

FastAPI + GenAI + Agentic AI Architecture (2026)

Status

There is **no single official file structure** for “FastAPI + GenAI + Agentic AI.”

This structure is **official-aligned and industry-ready**, based on:

- FastAPI larger-app structure with packages and `APIRouter`
- Pydantic AI reusable agents
- OpenAI Agents SDK concepts: agents, tools, handoffs, state, guardrails, tracing
- Google Gen AI SDK and Google ADK provider/agent patterns
- LangChain/LangGraph, LlamaIndex, CrewAI, Microsoft Agent Framework, Haystack, DSPy, LiteLLM, MCP, and observability ecosystem compatibility

FastAPI officially recommends modular larger apps using multiple files, packages, and `APIRouter`.

Pydantic AI describes agents as reusable objects similar to FastAPI apps or routers. OpenAI describes agents as applications that plan, call tools, collaborate across specialists, and keep state. LangChain’s own docs distinguish between agent frameworks like LangChain, runtimes like LangGraph, and harnesses like Deep Agents.

Design Goals

- Model agnostic
- Framework agnostic at the AI layer
- FastAPI-native
- Agent-ready
- Production-friendly
- Scalable from single-agent to multi-agent systems
- Compatible with OpenAI, Gemini, Anthropic, Bedrock, Azure OpenAI, Groq, Ollama, and future providers

Recommended Project Structure

```
app/  
├─ main.py  
├─  
├─ core/  
│   └─ config.py  
│   └─ logging.py
```

```
| | security.py
| | exceptions.py
|
| api/
| | deps.py
| | v1/
| | | router.py
| | | chat.py
| | | agents.py
| | | retrieval.py
| | | health.py
|
| schemas/
| | chat.py
| | agents.py
| | retrieval.py
| | common.py
|
| services/
| | chat_service.py
| | agent_service.py
| | retrieval_service.py
| | eval_service.py
|
| ai/
| | models/
| | | base.py
| | | factory.py
| | | providers.py
| |
| | agents/
| | | base.py
| | | workflows.py
| | | state.py
| |
| | tools/
| | | registry.py
| | | builtins.py
| |
| | prompts/
| | | system.md
| | | loader.py
| |
| | rag/
| | | embeddings.py
| | | retriever.py
```

```

|   |   |— vector_store.py
|   |   |— reranker.py
|   |   |— chunking.py
|   |
|   |— memory.py
|   |— guardrails.py
|   |— evals.py
|
|— infra/
|   |— db.py
|   |— cache.py
|   |— queue.py
|   |— storage.py
|   |— observability.py
|
|— tests/
|   |— unit/
|   |— integration/
|   |— evals/
|
|— scripts/
|   |— ingest.py
|   |— run_evals.py
|
|— migrations/
|
|— pyproject.toml
|— Dockerfile
|— docker-compose.yml
|— .env.example
|— README.md

```

Layer Responsibilities

Layer	Purpose
api/	FastAPI routers, HTTP endpoints, auth dependencies, response formatting
schemas/	Pydantic request/response models, DTOs, validation schemas
services/	Application use cases, business workflows, transaction boundaries
ai/models/	Model/provider abstraction layer
ai/agents/	Agent workflows, state, handoffs, planning, execution

Layer	Purpose
<code>ai/tools/</code>	Function tools, search tools, DB tools, external APIs, MCP tools
<code>ai/rag/</code>	Embeddings, chunking, retrieval, re-ranking, vector storage
<code>ai/memory.py</code>	Conversation, session, long-term, and user memory
<code>ai/guardrails.py</code>	PII checks, prompt-injection checks, output validation, safety filters
<code>infra/</code>	DB, cache, queue, storage, tracing, monitoring

`api/` must not contain:

LLM calls
Agent orchestration
Database logic

Dependency Direction

Required:

`api → services → ai / infra`

Avoid:

`api → OpenAI SDK`
`api → Gemini SDK`
`api → LangChain / LangGraph directly everywhere`
`api → Vector DB`

Correct pattern:

`api/chat.py → services/chat_service.py → ai/models/*`
`api/agents.py → services/agent_service.py → ai/agents/*`
`api/retrieval.py → services/retrieval_service.py → ai/rag/*`

Model-Agnostic Contract

app/ai/models/base.py

```
from typing import Protocol, Sequence, Any

class LLMClient(Protocol):
    async def generate(
        self,
        messages: Sequence[dict[str, Any]],
        *,
        model: str | None = None,
        temperature: float = 0.2,
        tools: list | None = None,
    ) -> dict:
        ...
```

All providers implement this contract.

Benefits:

- Provider swapping
- Easier testing
- Reduced lock-in
- Cleaner architecture

Example provider implementations:

```
OpenAI
Gemini
Anthropic
Azure OpenAI
Bedrock
Groq
Ollama
LiteLLM gateway
```

LiteLLM is especially useful when you want one unified interface or gateway for many model providers. Its docs describe a unified interface for 100+ LLMs, including OpenAI, Anthropic, Vertex AI, Bedrock, Azure, and others.

Agent Framework Ecosystem Fit

The structure **does consider LangChain/LangGraph and other 2026 agent frameworks**. They should live inside `ai/agents/`, not inside FastAPI routes.

Framework / Library	Best Fit In This Structure	Use When
Pydantic AI	<code>ai/agents/</code> , <code>ai/tools/</code> , <code>schemas/</code>	Type-safe Python agents, dependency injection, Pydantic-native apps
OpenAI Agents SDK	<code>ai/agents/</code> , <code>ai/tools/</code> , <code>ai/guardrails.py</code>	OpenAI-first agents, handoffs, tools, tracing, guardrails
LangChain	<code>ai/agents/</code> , <code>ai/tools/</code>	Quick agent apps, standard abstractions, integrations
LangGraph	<code>ai/agents/workflows.py</code> , <code>ai/agents/state.py</code>	Durable, stateful, long-running, human-in-the-loop, graph workflows
LlamaIndex	<code>ai/rag/</code> , <code>ai/agents/</code>	RAG, indexing, query engines, data agents, document workflows
CrewAI	<code>ai/agents/</code>	Multi-agent crews, role-based collaboration, flows
Google ADK	<code>ai/agents/</code> , <code>ai/tools/</code> , <code>ai/evals.py</code>	Google/Gemini ecosystem, enterprise-scale multi-agent systems
Microsoft Agent Framework	<code>ai/agents/</code> , <code>ai/tools/</code> , <code>ai/memory.py</code>	Microsoft/Azure ecosystem, graph workflows, MCP, state, middleware
Haystack	<code>ai/rag/</code> , <code>ai/agents/</code>	Modular RAG, pipelines, search, production LLM workflows
DSPy	<code>ai/rag/</code> , <code>ai/evals.py</code> , <code>ai/agents/</code>	Programmatic prompt/pipeline optimization, RAG/agent-loop optimization
MCP	<code>ai/tools/registry.py</code> , <code>infra/</code>	Standardized external tool/resource/prompt integration
Langfuse / LangSmith / OpenTelemetry	<code>infra/observability.py</code> , <code>ai/evals.py</code>	Tracing, debugging, monitoring, evaluation

LangChain's docs explicitly separate agent frameworks, runtimes, and harnesses. They describe LangGraph as a runtime for long-running, stateful agents with durable execution, streaming, human-in-the-loop, persistence, and low-level control.

LangGraph's own repository describes it as a low-level orchestration framework for stateful agents with durable execution, human-in-the-loop, comprehensive memory, debugging, and production deployment support.

Agent Layer

`ai/agents/` may contain:

```
Pydantic AI
OpenAI Agents SDK
LangChain
LangGraph
CrewAI
Google ADK
Microsoft Agent Framework
LlamaIndex agents
Haystack agents
Custom orchestration
```

Use this layer for:

- Single-agent workflows
- Multi-agent workflows
- State transitions
- Handoffs
- Human approval
- Planning/execution loops
- Graph-based workflows

OpenAI Agents SDK tracing includes LLM generations, tool calls, handoffs, guardrails, and custom events. Google ADK is documented as an open-source agent development framework for building, debugging, evaluating, and deploying reliable agents at enterprise scale.

Microsoft Agent Framework supports agents, tools, MCP servers, graph workflows, state management, memory/context providers, middleware, and telemetry. Microsoft also states that it is the successor to Semantic Kernel and AutoGen; AutoGen itself is now described as being in maintenance mode for new projects.

RAG Layer

`ai/rag/` owns the retrieval pipeline:

```
documents → chunking → embeddings → vector store → retrieval → reranking →  
answer generation
```

Good fits:

```
LlamaIndex  
Haystack  
LangChain retrievers  
Custom RAG  
Vector DB SDKs  
Hybrid search  
Rerankers
```

LlamaIndex's current documentation is organized around building agents, RAG pipelines, indexing, retrievers, vector stores, observability, and evaluation. Haystack describes itself as an open-source AI framework for production-ready AI agents, RAG applications, and scalable multimodal search systems.

Tooling, MCP, and External Integrations

`ai/tools/` should own:

```
function tools  
search tools  
database tools  
API tools  
MCP client wrappers  
tool schemas  
tool permissions
```

MCP should not replace your internal tool registry. Treat MCP as an integration protocol under `ai/tools/registry.py` or `infra/`.

MCP's official architecture describes a JSON-RPC-based protocol with tools, resources, prompts, notifications, lifecycle management, and transport/authorization concerns. MCP's tool specification also notes that tool invocations should have human-in-the-loop denial capability for trust and safety.

Observability and Evals

Keep observability in:


```
infra/observability.py
```

Keep AI evaluation logic in:

```
ai/evals.py  
tests/evals/
```

Good fits:

```
OpenAI tracing  
LangSmith  
Langfuse  
OpenTelemetry GenAI semantic conventions  
Pydantic Evals  
LlamaIndex evals  
DSPy optimizers/evaluators  
promptfoo / custom evals
```

Langfuse describes LLM observability as tracing, monitoring latency, tracking costs, and debugging across OpenAI, LangChain, LlamaIndex, and more. OpenTelemetry now has GenAI semantic conventions covering spans, metrics, and events for GenAI clients, MCP, and provider-specific conventions.

DDD Position

DDD is **optional**.

Do **not** add `domain/` by default just because the app uses FastAPI, GenAI, RAG, or agents.

Add DDD only when the app has meaningful business-domain complexity:

```
business rules  
domain invariants  
approval workflows  
financial calculations  
policy rules  
multi-step enterprise processes  
multiple bounded contexts
```

Default structure:

```
api/  
schemas/  
services/  
ai/  
infra/  
tests/
```

Optional DDD extension:

```
app/  
├── domain/  
│   ├── entities.py  
│   ├── value_objects.py  
│   ├── aggregates.py  
│   ├── events.py  
│   └── errors.py  
├── repositories/  
│   ├── base.py  
│   └── conversation_repository.py
```

With DDD:

```
api → services → domain / ai / infra
```

Rule:

Simple/medium GenAI app:
No domain/ folder.

Complex enterprise app:
Add domain/ when business rules, aggregates, or bounded contexts become important.

Corrected 2026 Position

The original structure is valid.

The only correction is this:

Do not say the structure is “official.”
Say it is “official-aligned.”

And this:

Explicitly document LangChain/LangGraph, LlamaIndex, CrewAI, Google ADK, Microsoft Agent Framework, Haystack, DSPy, LiteLLM, MCP, and observability tools as implementations inside the existing layers.

No required folder changes.

Optional future split if the project grows:

```
ai/agents/workflows.py → ai/agents/langgraph_workflows.py  
ai/models/providers.py → ai/models/openai_provider.py, google_provider.py, etc.  
infra/observability.py → infra/observability/  
ai/tools/registry.py → ai/tools/mcp.py + ai/tools/registry.py
```

Final Recommendation

For 2026, the safest architecture is:

```
FastAPI  
+  
Pydantic  
+  
Service Layer  
+  
Provider Abstraction  
+  
Agent Layer  
+  
Tools / MCP  
+  
RAG  
+  
Memory  
+  
Guardrails  
+
```

Evals
+
Observability
+
Infrastructure Adapters

This gives a scalable, model-agnostic, production-ready foundation while staying aligned with official framework guidance and the current GenAI/agent AI ecosystem.

References

- FastAPI — Bigger Applications / APIRouter.
- Pydantic AI — Agents core concepts.
- OpenAI — Agents SDK guide and tracing.
- Google Gen AI SDK and Google ADK.
- LangChain / LangGraph framework-runtime-harness guidance.
- LlamaIndex documentation.
- CrewAI documentation.
- Microsoft Agent Framework and AutoGen status.
- Haystack documentation.
- DSPy documentation.
- LiteLLM documentation.
- MCP architecture and tools specification.
- Langfuse and OpenTelemetry GenAI observability.